

# Machine Learning in Production

*Week 9 · Module 2, Lectures 1–3*

CS 5090 · Software Engineering with Generative AI · Week 9 · Module 2, Lectures 1–3

Michigan Technological University · Fall 2026



# Lecture 1 · From Models to Systems

# From Models to Systems

*Why the model is the small part*

- Module 1 built the box. Module 2 builds everything around it.
- Module 1: the model · Module 2: the system · Module 3: the practice





# A running example: an automated transcription start-up

*We will return to this example all module*

- **The product**: upload audio, get an accurate, searchable transcript.
- **The model**: speech recognition. Impressive, and about 5% of the work.
- **The system**: user interface, accounts, upload, payment, database, cloud processing, logging, monitoring.
- The hard questions are all systems questions: what if it mishears a name? Who notices? Who fixes it? What does the customer see meanwhile?

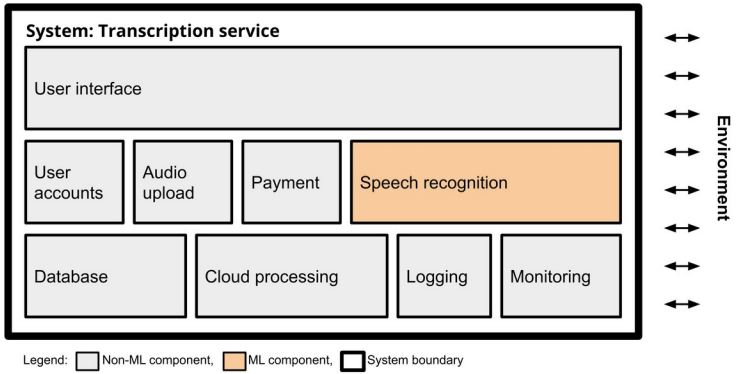



*Figure: Kästner, Machine Learning in Production, MIT Press 2025, ch. 2 (CC BY-NC-ND 4.0).*

# The model is a small box

*Ten years old and still the most-cited picture in the field*

- The tiny box in the middle is Module 1. Every other box is Module 2, and each one can take the product down on its own.

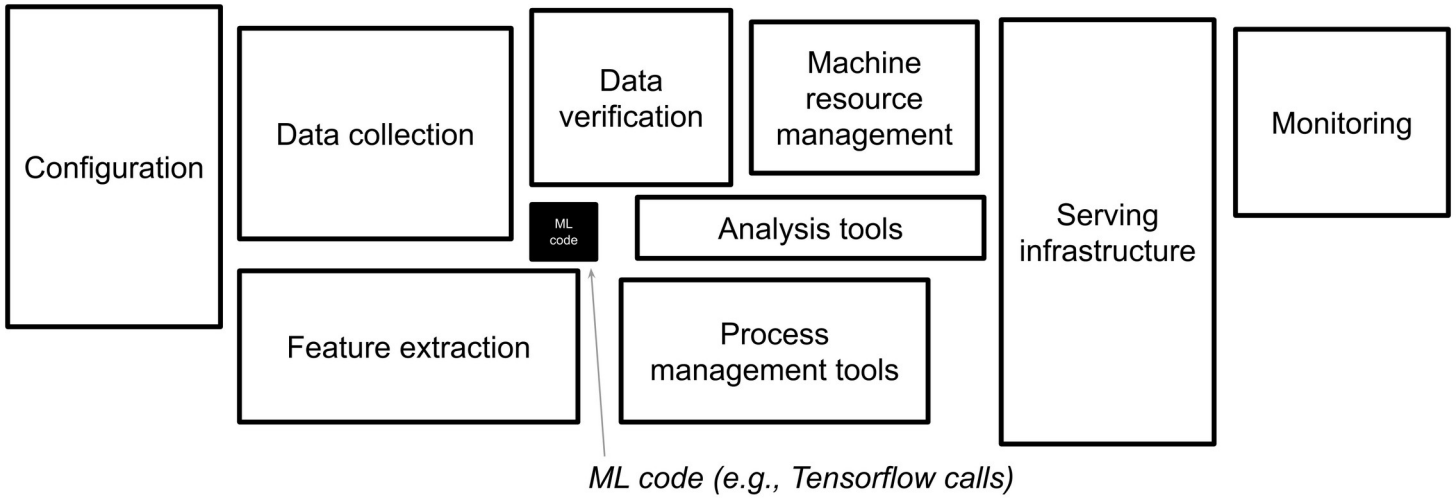



*Figure after Sculley et al., Hidden Technical Debt in Machine Learning Systems, NeurIPS 2015, Fig. 1; redrawn in Kästner 2025, ch. 2.*

# Nine stages, and the model is one of them

*The workflow around a model, as practised at Microsoft*

- Data-oriented stages: requirements, collection, cleaning, labeling.
- Model-oriented stages: feature engineering, training, evaluation, deployment, monitoring.
- Two feedback loops: evaluation and monitoring send you back to training, or all the way back to the data.

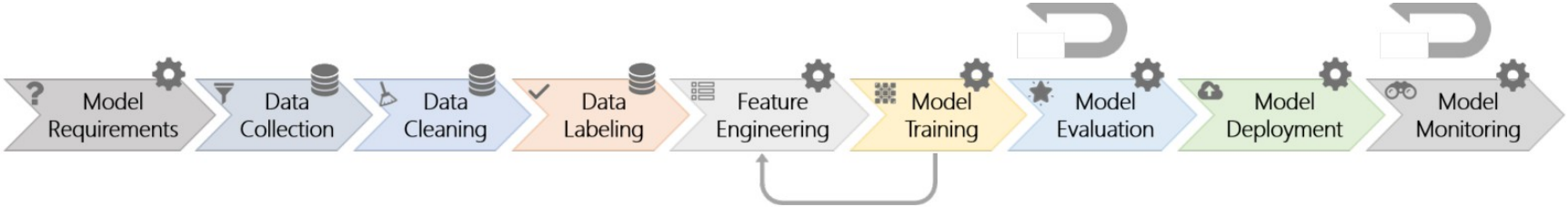



*Figure: Amershi et al., Software Engineering for Machine Learning: A Case Study, ICSE-SEIP 2019, Fig. 1.*

# Two professions, one product

*Different training, different instincts, same deadline*

- **Data scientists**: explore, iterate, optimise a metric. Home: the notebook (iterate fast, throw away).
- **Software engineers**: specify, structure, keep it running. Home: the codebase (review, test, maintain).
- Neither view alone ships a product.
- Most project failures are at the seam between them.
- SE · You will be the person who can speak both.

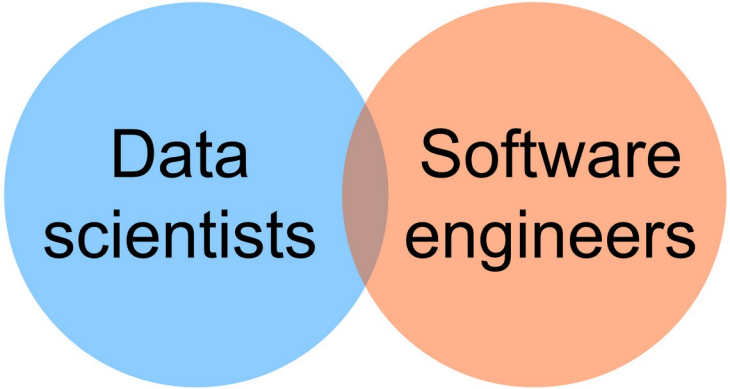



*Figure: Kästner 2025, ch. 1. Study: Amershi et al. 2019.*

# Collaboration challenges in ML-enabled systems

*What goes wrong at the seam, in the words of practitioners*

- Industry keeps replacing hand-written logic with learned components: at Snapchat, friend recommendation went from a round-robin ranker to an ML ranker, and from traditional retrieval to embedding-based (GNN) retrieval.
- The development process did not keep up: Agile, customised and unprincipled.
- Interview studies find the same three gaps: **lack of a standard process**, **unclear responsibility**, **lack of a system-wide view**.
- Also: poor documentation, model teams that assume no responsibility for testing the product, product teams with no planned system-level testing.





# A systems-engineering answer: the V-model

*Decompose on the way down; verify and validate on the way up*

- Left leg: system requirements → architecture → module design, down to coding.
- Right leg: unit testing → integration testing → system testing → acceptance testing, each level checked against the matching level on the left.
- Interdisciplinary, system-level, quality-focused: a standard for large projects (ISO/IEC/IEEE 15288) built to reduce complexity and risk.
- Compared with Agile, Spiral, and Waterfall it scores high on system safety, system validation, and clear boundaries and responsibility; low on speed to market and tolerance of changing requirements.

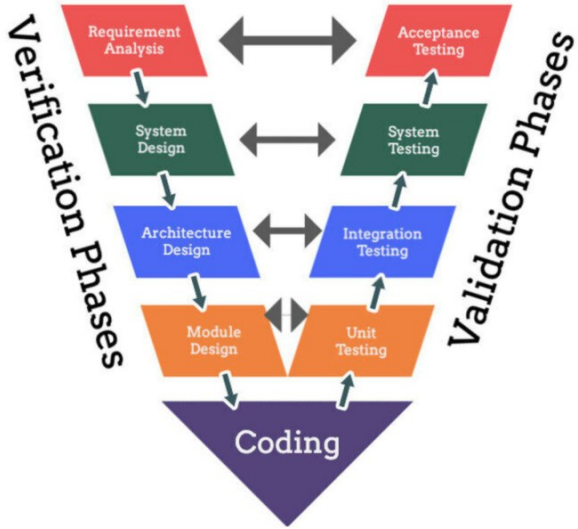



*Figure: Wu, An Exploratory Study of V-Model in Building ML-Enabled Software: A Systems Engineering Perspective, CAIN 2024.*

# V-model plus eleven systems-engineering functions

*The process input is an objective; the output is a decision to move to the next phase*

- Technical process, each phase: SE1 objectives → SE2 derived requirements → SE3 architecture → SE4 user scenarios, with SE5 V&V on every arrow.
- Management process alongside: SE6 interfaces · SE7 external events · SE8 resources · SE9 risk · SE10 documentation · SE11 milestone reviews.
- Systems engineering is additional work; complex systems do not ship without it.

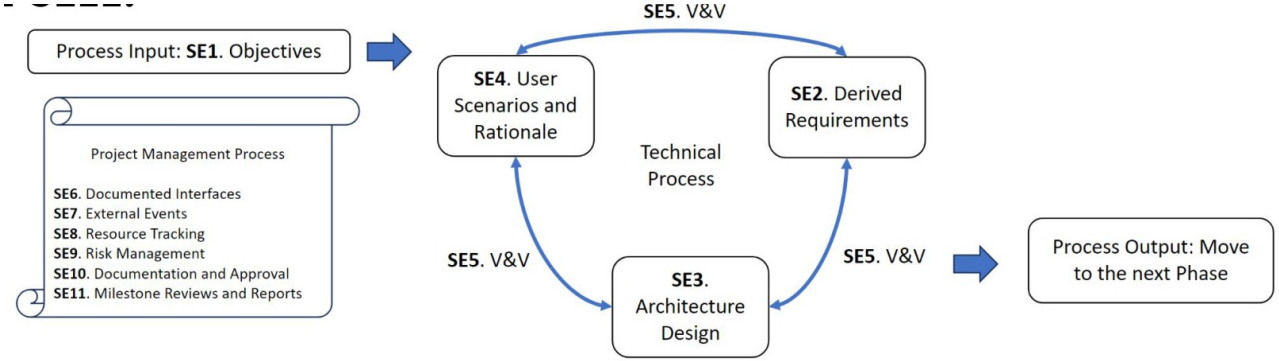



*Figure: Wu, CAIN 2024, after ISO/IEC/IEEE 15288 and the MathWorks systems-engineering lectures.*

# What practitioners said: eight propositions

*Interviews with 11 practitioners from 9 companies (Wu, CAIN 2024)*

| # | Proposition | Lecture |
|---|---|---|
| PR1 | System requirements, actively maintained, with ML and non-ML owners participating | L3 |
| PR2 | System-level architecture: elements, interfaces, responsibilities, alternatives, expected performance | L4 |
| PR3 | Risks from ML uncertainty identified and mitigated on purpose | L3, L4 |
| PR4 | Requirements and detailed design of the ML component, with data and infra owners | L3 |
| PR5 | Data is a separate component: own requirements, design, validation, monitoring | L5 |
| PR6 | V&V at system, subsystem, and component level, with named owners | L5 |
| PR7 | Follow the V: layered decomposition, continuous V&V and risk management, responsibility boundaries | L7 |
| PR8 | Documentation at every level, approved by owners, tracked, shared terminology | L7 |





# What makes ML systems different

*Five properties that break ordinary software assumptions*

| Property | What it means |
|---|---|
| No specification | the model is induced from data, not written from a spec |
| Mistakes are certain | not a bug to fix, a rate to design around |
| The world moves | data drift silently degrades a model that never changed |
| Data is code | the training set is a dependency you must version |
| Entanglement | changing anything changes everything downstream (CACE) |





# The pipeline is part of the system too

*A model is produced by code that must be engineered like any other code*

- The pipeline that builds the model (collect, clean, label, features, train, evaluate) ships with the product.
- Deployment and monitoring close the loop: a retrain is a job you run, not a notebook you remember.
- Cloud, scaling, and MLOps are the engineering of that loop. Lecture 4 is about it.

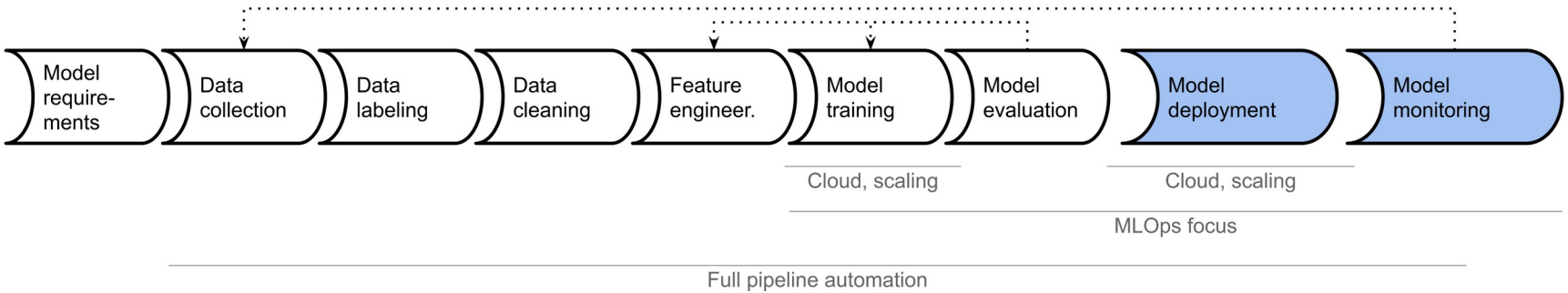



*Figure: Kästner 2025, ch. 2.*

# Coding view: your model in a product

*What Project 1 would need before anyone could use it*

- An inference service that answers in under a second.
- A way to send it repository context, and a token budget.
- Telemetry: was the suggestion accepted, edited, or discarded?
- A fallback for when the model is down or too slow.
- model (Module 1) → serving (latency, cost) → context (what to send) → feedback (did it help?)
- SE · GitHub Copilot is roughly 2% model and 98% this list.





# Key takeaways

- The model is one component; the product is everything around it.
- Most failures in ML products are systems failures, not model failures.
- ML breaks four software assumptions: specs, correctness, stability, and modularity.
- The pipeline that produces the model is a second system to engineer.
- Data scientists and software engineers both need the other's view; the V-model's decomposition, boundaries, and V&V are one answer to the seam.





# Readings & next time

- T2 (Kästner), chapter 2, "From Models to Systems": this lecture in prose. Chapter 3 if you want the ML refresher.
- Before Wednesday: pick a product you use that has ML inside and sketch its non-ML parts.
- Wednesday: when ML is the right tool, and what "better" means before you build.
- Friday: Project 1 bundle due in full.







# Lecture 2 · When to Use ML, and What For

# When to Use ML, and What For

*Fit, goals, and measurement*

- Two decisions that come before any code: should this be machine learning at all, and how will we know whether it worked?
- Both are routinely skipped, and both sink projects.





# When ML is the right tool

*All five should hold. If several do not, write rules instead*

| Condition | Why it matters |
|---|---|
| There is a pattern | and it is too complex to write down by hand |
| You have the data | enough of it, representative of real use |
| Mistakes are tolerable | or you can catch and recover from them |
| The task keeps changing | so a fixed rule set would rot |
| The value beats the cost | including the cost of operating it forever |





# How bad is a mistake?

*The question that decides your whole design*

| | rare | common |
|---|---|---|
| catastrophic | × | × |
| costly | × | × |
| annoying | ok | × |
| invisible | ok | ok |

*ok = machine learning is a reasonable fit · × = needs guardrails, or a different approach.*





# Nobody buys accuracy

*Booking.com: better offline models did not mean better business results*

- 23 successful model improvements at Booking.com, each better offline.
- Relative improvement in model performance (x) against relative change in conversion rate (y): no correlation.
- Reasons the authors found: saturation, diminishing returns, uncanny-valley effects, and proxy metrics that stopped tracking the goal.
- A better model is not the same thing as a better product.

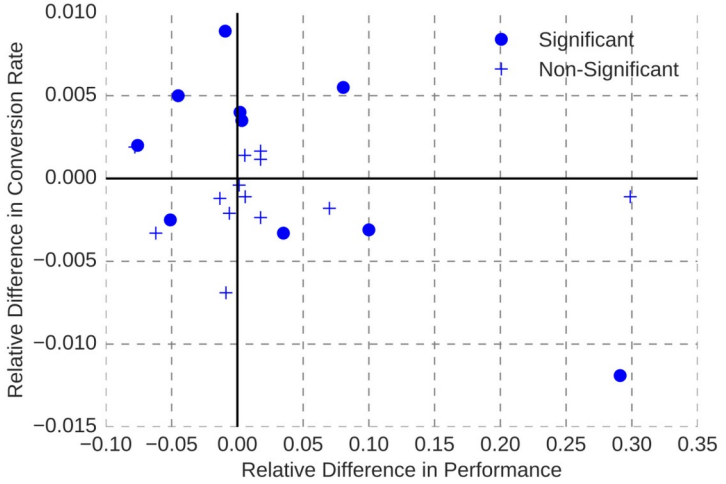



*Figure: Bernardi, Mavridis & Estevez, 150 Successful Machine Learning Models: 6 Lessons Learned at Booking.com, KDD 2019; reproduced in Kästner 2025, ch. 2.*

# Goals come in layers

*Accuracy is at the bottom. Nobody buys accuracy*

- **Organisational goals**: revenue, retention, cost.
- **Product goals** and **leading indicators**: signups, usage, churn.
- **User goals**: did the user get what they came for?
- **Model goals**: accuracy, F1, pass@1.
- Read top-down when you design; bottom-up when you justify.

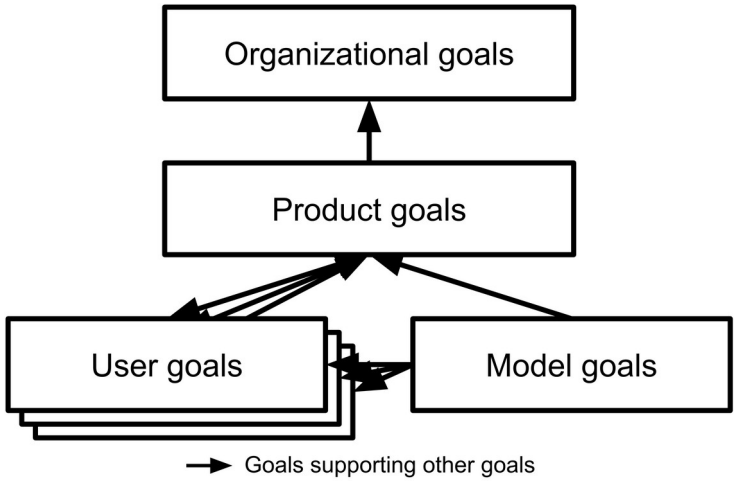



*Figure: Kästner 2025, ch. 5.*

# A prediction is not a decision

*From the model's output to the thing the business wants*

- The model **answers** a question; the question **informs** a decision; the decision **supports** a goal.
- A chatbot that classifies intent well does not help if the decision taken on the intent is wrong.
- Design the decision first, then ask what prediction would improve it.

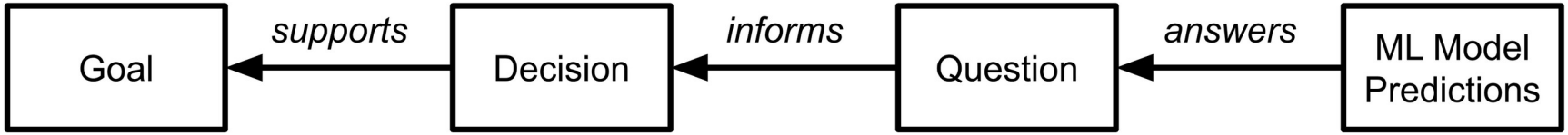



*Figure: Kästner 2025, ch. 5.*

# Measurement, briefly

*Three questions to ask of any metric*

- **Construct validity**: are we measuring the thing we care about, or a convenient proxy?
- **Reliability**: would we get the same number tomorrow, with a different sample?
- **Cost**: what does it take to collect, and will anyone actually collect it?
- The streetlight effect: measuring where the light is, not where the keys are.
- Goodhart's law (Week 8): once a measure is a target, it stops being a good measure. Every metric will be gamed, by your team, your users, or your model.

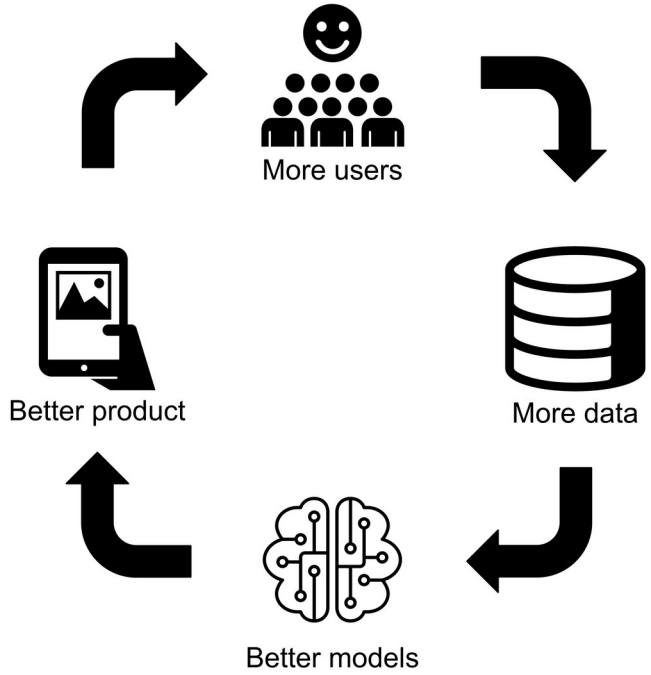



*Figure: Kästner 2025, ch. 5. Goodhart's law as phrased by Strathern 1997.*

# The business case

*Machine learning makes predictions cheap. That is the whole economic argument*

- Cheaper predictions mean you use prediction where you never could before.
- But judgement about what to DO with a prediction stays expensive, and the cost of a wrong action does not fall at all.
- So the value is in the decision, not in the prediction.
- The flywheel: more users → more data → better models → better product → more users.
- SE · A code suggestion is cheap; a wrong merged patch is not.




*Flywheel: Kästner 2025, ch. 1. Argument: Agrawal, Gans & Goldfarb, Prediction Machines, 2018.*

# Coding view: goals for a code assistant

*Four layers, made concrete*

| Layer | For a coding assistant |
|---|---|
| Organisational | features shipped per engineer, defect escape rate |
| Leading indicator | daily active use, suggestions per session |
| User outcome | acceptance rate, and survival of accepted code after a week |
| Model quality | pass@1 on a held-out internal benchmark |





# Key takeaways

- Use ML when there is a pattern, data, tolerance for mistakes, and a changing task.
- How bad a mistake is decides your architecture, not your model choice.
- A better model is not a better product: Booking.com's 23 models.
- Goals stack: organisational → product → user → model; prediction → question → decision → goal.
- Ask of every metric: valid, reliable, affordable, and expect it to be gamed.
- Reading: T2 chapters 4 and 5. Homework: one leading and one lagging indicator for your Project 2 topic. Friday: requirements and planning for mistakes; Project 1 due.







# Lecture 3 · Requirements and Planning for Mistakes

# Requirements and Planning for Mistakes

*The model will be wrong*

- Requirements for a system whose component is only probably right.
- What you build so that being wrong is survivable.
- Second half (⚙): what this does to specification when the implementer is a model.





# The world and the machine

*Your requirements live in the world; your software lives in the machine*

- What the user wants happens in the WORLD. What you can build happens in the MACHINE.
- Software only sees input data from sensors and people, and only produces output data for displays and actuators.
- They are connected only by **assumptions** about the environment.
- Most failures are broken assumptions, not broken code. Write the assumptions down; they are requirements too.
- Example: the person fell (world) · a fall makes this accelerometer pattern (assumption) · the classifier fires (machine).

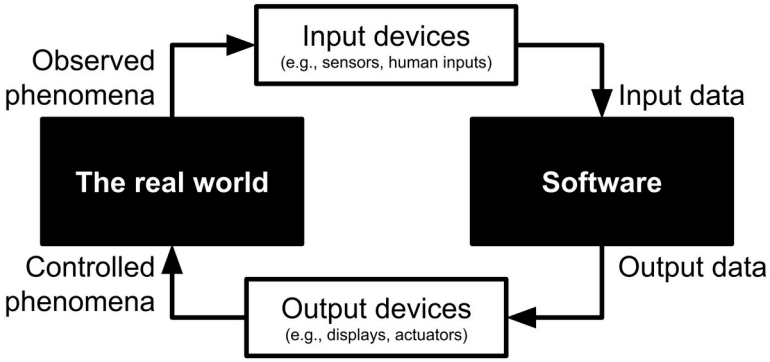
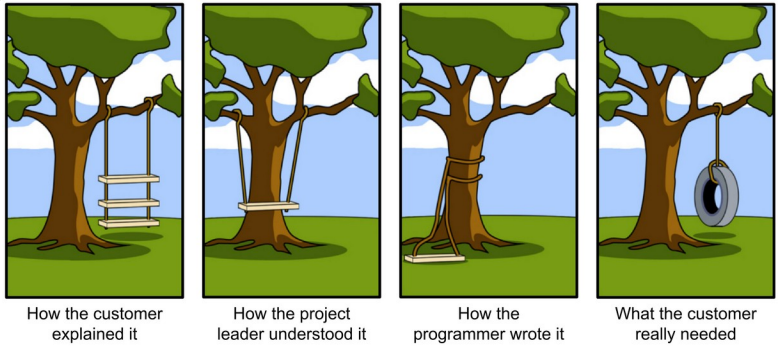



*Figure: Kästner 2025, ch. 6, after Jackson, The World and the Machine, ICSE 1995.*

# Untangling requirements

*Four different things people call "requirements"*

- **System requirements**: what the product must achieve in the world.
- **User requirements**: what a specific person needs to accomplish.
- **Environment assumptions**: what must be true for your design to work. The row that gets skipped.
- **Component specifications**: what the model, the API, the UI must each do.
- Confusing these is how a team builds the wrong thing efficiently.

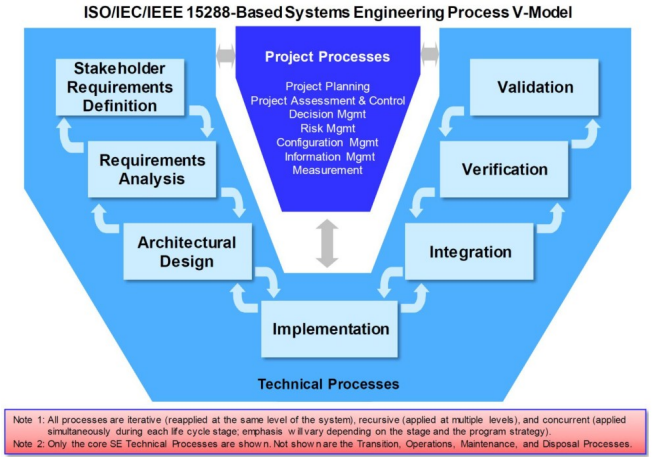



*Cartoon: the classic "tree swing," as reproduced in Kästner 2025, ch. 6.*

# Requirements in the V-model: PR1, PR3, PR4

*What practitioners asked for, and what it costs*

- Challenges reported: vague specifications; unrealistic expectations from managers; conflicts over code quality; thin infrastructure support.
- **PR1**: system requirements, actively maintained as they change, with the owners of both ML and non-ML components participating.
- **PR4**: the ML component gets its own requirements and design: interfaces, alternatives, expected performance, written with the data and infra owners.
- **PR3**: risks from ML uncertainty are identified and mitigated on purpose (SE9), not discovered in production.
- Gains: boundaries, responsibility, risk management, V&V. Costs: less agility, more effort.




*Wu, An Exploratory Study of V-Model in Building ML-Enabled Software, CAIN 2024; figure after ISO/IEC/IEEE 15288.*





# Mistakes are not bugs

*You cannot fix your way to a model that is always right*

- Every model has a residual error rate, permanently.
- More data and more parameters move it, never to zero.
- So the design question is not "how do we prevent mistakes." It is "what happens to the user when one occurs."
- SE · The same reasoning applies to every AI coding tool you will use.

| Version | error rate % |
|---|---|
| v1 | 22.0 |
| v2 | 14.0 |
| v3 | 9.0 |
| v4 | 7.0 |
| v5 | 6.2 |





# Designing for failure

*Six patterns, cheapest first*

- **Make it undoable**: a wrong action the user can reverse costs almost nothing.
- **Lower the forcefulness**: suggest instead of act; ask instead of assume. The figure: Gmail suggests three replies and sends none.
- **Add a guardrail**: a cheap non-ML check that vetoes obvious nonsense.
- **Ask a human**: route the low-confidence cases to a person.
- **Use redundancy**: two different methods must agree before acting.
- **Degrade gracefully**: a defined, safe behaviour when the model is unavailable.

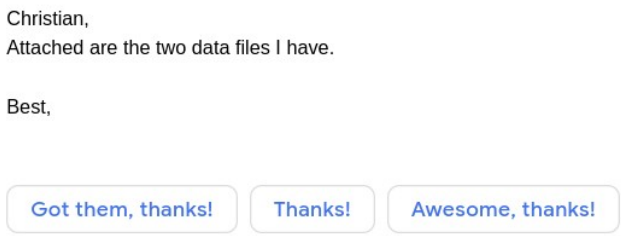



*Figure: Kästner 2025, ch. 7. Patterns: Hulten 2018; Kästner 2025.*

# Guardrails outside the model

*Platform screen doors: the mitigation is not a better train*

- The train's door sensor is an ML-free guardrail; the screen door is a second, independent one.
- Safety came from the system around the component, not from making the component perfect.
- Ask of every ML component: what is the screen door?

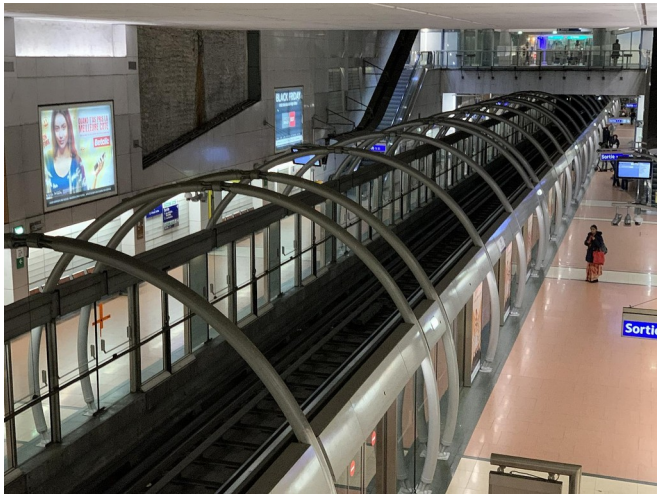


*Photo: Kästner 2025, ch. 7.*

# Risk analysis you can actually do

*Work backwards from the harm, with a fault tree*

- Start from the harm (top event) and decompose with AND / OR down to basic events: ML model mistake, camera defective, poor light, sensor fails.
- The mitigation (green) adds an independent safety system, turning an OR into an AND: the model mistake alone no longer suffices.
- Most controls are non-ML: a sensor, a confirmation dialog, a rate limit, a human review step.

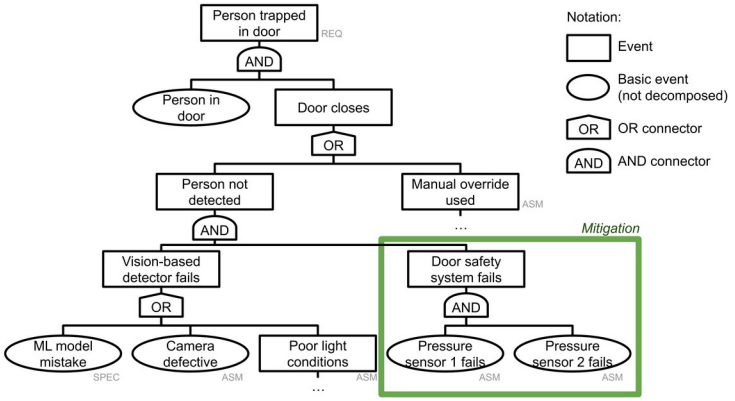



*Figure: Kästner 2025, ch. 7.*

# Coding view: risk for a coding agent

*The same analysis, for a tool that edits your repo*

| Column | For a coding agent |
|---|---|
| Harm | broken production, leaked secret, silent logic error |
| Hazard | an unreviewed patch reaching the main branch |
| Cause | agent edits the wrong file; tests too weak to notice |
| Control | sandbox, branch-only writes, mandatory human review, full test run |

*agent proposes (unconstrained) → sandbox + tests (automatic) → human review (accountable)*
*SE · None of the controls is "a better model."*





# ⚙️ What a usable spec contains

*The five things a model, or a new colleague, cannot infer*

| Part | Contents |
|---|---|
| Interfaces | exact signatures, types, error cases |
| Behaviour | what happens for normal, edge, and invalid input |
| Constraints | performance, compatibility, what must not change |
| Examples | two or three concrete input/output pairs |
| Non-goals | what you deliberately are NOT building |





# ⚙️ Spec-first, in practice

*A workflow you can adopt this week*

- Write the interface and the docstring yourself, by hand.
- Write two examples and one edge case as tests, also by hand.
- THEN ask for the implementation.
- The tests you wrote are now an oracle you trust (Week 8).
- "add caching" (ambiguous) → interface + docstring (constrained) → + tests you wrote (verifiable)
- SE · You have moved the model from author to implementer.





# Key takeaways

- Requirements live in the world; software lives in the machine; assumptions connect them.
- Distinguish system, user, environment, and component requirements, or build the wrong thing; the V-model's PR1 and PR4 make both ML and non-ML owners write and maintain them.
- A model's residual error rate never reaches zero, so design for the errors that remain.
- Undo, lower forcefulness, guardrails, human review, redundancy, graceful degradation.
- Analyse risk backwards from the harm with a fault tree; most good controls are not ML.
- ⚙ Specs need interfaces, behaviour, constraints, examples, and non-goals; write the tests first.
- Project 1 bundle is due tonight. Monday: architecture, deployment, and the pipeline around the model.



